# Estudo Dirigido 01 – Pizzaria Caótica

Documentação do Código e Estrutura do Projeto

CC7N Rafael Wernesbach
Rafael Bassul

19 de março de 2026

## 1 Introdução

Este documento apresenta a documentação completa do projeto da pizzaria concorrente, seguindo a organização solicitada no PDF da atividade. O código foi dividido em quatro arquivos principais: `main.py`, `pizzaria.py`, `interface.py` e `graph.py`. Em conjunto, esses arquivos implementam a simulação da produção e entrega de pizzas com múltiplas threads, controle de recurso limitado por semáforo, sincronização por condição e visualização do estado da execução.

O projeto modela um cenário produtor-consumidor em que chefs produzem pizzas premium a partir da busca por números primos grandes, enquanto entregadores consomem os pedidos prontos e realizam a entrega. Além da simulação, o projeto também possui uma interface gráfica para acompanhar o estado interno e um script separado para medição de desempenho e geração de gráficos.

## 2 Visão Geral da Arquitetura

| **Arquivo** | **Responsabilidade principal** |
|---|---|
| `main.py` | Ponto de entrada da aplicação. Lê os parâmetros da simulação, inicia a interface gráfica e dispara a execução concorrente em uma thread separada. |
| `pizzaria.py` | Núcleo da lógica concorrente. Define fila, semáforo, condição, estado compartilhado, funções auxiliares e as classes `Chef` e `Entregador`. |
| `interface.py` | Implementa a interface em Tkinter, exibindo em tempo real o estado dos chefs, dos entregadores e das estatísticas gerais da simulação. |
| `graph.py` | Executa a simulação em diferentes configurações, mede o tempo total de execução, calcula o speedup e gera os gráficos de desempenho. |

# 3 Etapa 1 – Planejamento e Definição do Problema

## 3.1 Entrada do sistema

As entradas principais do sistema são:

- quantidade de chefs;
- quantidade de entregadores;
- quantidade de pizzas que cada chef deve produzir.

Esses valores são lidos no arquivo `main.py` por meio do terminal, antes da criação da interface gráfica e do início da simulação.

## 3.2 Saída esperada

As saídas esperadas do sistema são:

- mensagens no terminal indicando a produção e a entrega das pizzas;
- atualização visual em tempo real da situação dos chefs e entregadores na interface gráfica;
- medições de tempo de execução e speedup no script `graph.py`;
- um gráfico comparando tempo total e speedup em função do número de threads.

## 3.3 Recursos limitados

O principal recurso limitado do sistema é o forno, modelado por um semáforo. Na implementação atual, o forno foi criado com:

```
threading.Semaphore(2)
```

Isso significa que apenas dois chefs podem usar o forno simultaneamente. O PDF sugere o uso de `Semaphore(3)`; portanto, a versão atual está conceitualmente correta, mas com uma capacidade diferente da sugerida no enunciado.

## 3.4 Produtores, consumidores e fila

Os produtores são os chefs, pois geram pizzas e colocam os pedidos prontos na fila compartilhada. Os consumidores são os entregadores, que retiram pizzas da fila e simulam a entrega.

Na implementação atual, a fila foi criada como:

```
Queue(maxsize=100)
```

O enunciado sugere uma fila na faixa de 30 a 50 itens. Assim, o projeto já utiliza uma fila limitada, mas o valor atual pode ser ajustado caso seja necessário aderir de forma literal à recomendação do PDF.

### 3.5 Configurações de teste

Para atender ao enunciado, as configurações mínimas recomendadas para medição são:

- 1 chef + 1 entregador;
- 2 chefs + 2 entregadores;
- 4 chefs + 4 entregadores;
- 8 chefs + 8 entregadores, ou o máximo suportado pela máquina.

O arquivo `graph.py` já automatiza os testes de desempenho exatamente nas combinações pedidas no PDF. Na versão atual, o script executa as configurações 1+1, 2+2, 4+4 e 8+8, sempre somando chefs e entregadores para calcular o total de threads analisado nos gráficos. Para que a comparação de speedup seja justa, o experimento mantém fixa a carga total de trabalho em 120 pizzas, distribuindo essa quantidade entre os chefs de cada teste.

### 3.6 Mecanismos de sincronização

Os mecanismos de sincronização adotados no projeto são:

- **Semaphore**: usado para controlar o acesso concorrente ao forno;
- **Condition**: usado para coordenar a comunicação entre chefs e entregadores quando uma pizza fica pronta e é inserida na fila;
- **Lock**: usado para proteger o dicionário de estado compartilhado que abastece a interface.

## 4 Etapa 2 – Implementação

Esta etapa documenta detalhadamente cada arquivo do projeto, explicando seu papel, suas estruturas internas e a forma como eles se relacionam.

### 4.1 `main.py`

O arquivo `main.py` funciona como ponto de entrada da aplicação. Ele organiza a inicialização do sistema e garante que a interface gráfica e a simulação concorrente funcionem ao mesmo tempo.

#### 4.1.1 Importações

Esse arquivo importa:

- `tkinter` para a criação da janela principal;
- `Interface` do arquivo `interface.py`;
- `iniciar_simulacao` do arquivo `pizzaria.py`;
- `threading` para executar a simulação sem bloquear a interface;
- `signal` e `sys` para tratar interrupção por teclado.

### 4.1.2 Função `signal_handler`

Essa função recebe o sinal de interrupção e encerra o programa de maneira simples. Ela imprime uma mensagem no terminal e chama `sys.exit(0)`.

### 4.1.3 Fluxo principal

No bloco `if __name__ == "__main__":`

1. o programa registra o tratador de `SIGINT`;
2. solicita ao usuário a quantidade de chefs;
3. solicita a quantidade de entregadores;
4. solicita a quantidade de pizzas por chef;
5. cria a janela principal do Tkinter;
6. instancia a classe `Interface`;
7. cria uma thread separada para executar `iniciar_simulacao`;
8. inicia o loop principal da interface com `root.mainloop()`.

### 4.1.4 Função do arquivo no sistema

Esse arquivo faz a integração entre interface e simulação. A decisão de executar a simulação em outra thread é importante porque evita o travamento da interface enquanto chefs e entregadores trabalham em paralelo.

## 4.2 `pizzaria.py`

O arquivo `pizzaria.py` concentra a lógica principal do projeto. Ele implementa o problema produtor-consumidor, a restrição de acesso ao forno, o acompanhamento do estado global e a criação das threads.

### 4.2.1 Estruturas globais

As principais estruturas globais são:

- `fila`: fila compartilhada de pizzas prontas;
- `forno`: semáforo que limita quantos chefs podem usar o forno ao mesmo tempo;
- `condition`: variável de condição usada para acordar entregadores quando uma pizza fica pronta;
- `estado_lock`: lock que protege as alterações no dicionário `estado`;
- `estado`: dicionário com dados de chefs, entregadores, quantidade na fila, pizzas produzidas e pizzas entregues;
- `SENTINELA`: objeto especial usado para sinalizar o encerramento dos entregadores.

### 4.2.2 Função `eh_primo`

A função `eh_primo(x)` verifica se um número é primo. O algoritmo:

- rejeita valores menores que 2;
- trata separadamente o caso do número 2;
- elimina números pares;
- testa divisibilidade apenas por ímpares até a raiz quadrada de `x`.

No contexto da simulação, cada número primo encontrado representa uma pizza premium pronta.

### 4.2.3 Funções de atualização de estado

As funções `atualizar_estado_chef` e `atualizar_estado_entregador` atualizam o dicionário compartilhado que abastece a interface. Elas registram:

- o nome da thread;
- o status atual;
- a quantidade de pizzas feitas;
- a fila atual;
- contadores globais de produção e entrega.

Essas funções são importantes porque desacoplam a simulação da interface: a lógica concorrente atualiza apenas o estado, enquanto a interface apenas o lê.

### 4.2.4 Classe `Chef`

A classe `Chef` herda de `threading.Thread`. Cada objeto dessa classe representa um produtor.

O método `run()` executa o seguinte fluxo:

1. marca o chef como separando ingredientes;
2. entra em laço até produzir a quantidade desejada de pizzas;
3. informa que está esperando o forno;
4. entra na região protegida pelo semáforo;
5. gera um número grande aleatório;
6. testa se o número é primo;
7. se for primo, registra uma pizza pronta;
8. insere a pizza na fila compartilhada;
9. incrementa o contador global de pizzas produzidas;

10. chama `condition.notify()` para acordar uma thread consumidora;
11. ao final, marca seu estado como finalizado.

Esse comportamento representa corretamente o papel de produtor e mostra o uso combinado de semáforo, fila e condição.

#### 4.2.5 Classe `Entregador`

A classe `Entregador` também herda de `threading.Thread`. Cada instância representa um consumidor.

O método `run()` segue o fluxo abaixo:

1. marca o entregador como ocioso;
2. entra em um laço infinito;
3. espera enquanto a fila estiver vazia, usando `condition.wait()`;
4. remove uma pizza da fila quando houver item disponível;
5. verifica se o item recebido é a sentinela;
6. se for sentinela, encerra a thread;
7. se for pizza normal, imprime a entrega, atualiza o estado e dorme por um curto intervalo para simular o tempo de entrega;
8. incrementa o contador de pizzas entregues e volta ao estado ocioso.

Essa classe materializa o lado consumidor do padrão produtor-consumidor.

#### 4.2.6 Função `iniciar_simulacao`

Essa função organiza toda a execução concorrente:

1. recebe as quantidades de chefs, entregadores e pizzas por chef;
2. cria as threads correspondentes;
3. inicia primeiro os entregadores;
4. inicia os chefs;
5. aguarda o término de todos os chefs com `join()`;
6. envia uma sentinela para cada entregador;
7. acorda todas as threads consumidoras com `condition.notify_all()` para evitar que alguma permaneça bloqueada em `condition.wait()`;
8. aguarda o término de todos os entregadores;
9. retorna as listas de threads criadas.

Essa organização evita deadlock no encerramento e garante que todos os entregadores terminem de forma controlada, inclusive quando alguma thread consumidora está aguardando nova pizza no momento em que a produção termina.

### 4.2.7 Função do arquivo no sistema

Esse é o arquivo mais importante do projeto, pois define a semântica da concorrência e a coordenação entre os componentes.

## 4.3 `interface.py`

O arquivo `interface.py` implementa a visualização da simulação em tempo real utilizando Tkinter.

### 4.3.1 Classe `Interface`

A classe `Interface` recebe o objeto `root` da janela principal e constrói os componentes visuais.

No construtor `__init__`, o arquivo:

- define o título da janela;
- cria um rótulo superior para estatísticas gerais;
- cria uma área de texto para exibir os chefs;
- cria uma área de texto para exibir os entregadores;
- chama imediatamente o método `update()`.

### 4.3.2 Método `update`

Esse método realiza a atualização periódica da interface:

1. entra em região protegida por `estado_lock`;
2. copia os dados de chefs, entregadores e contadores;
3. atualiza o rótulo principal com pizzas produzidas, entregues e tamanho da fila;
4. limpa e reescreve a caixa de texto dos chefs;
5. limpa e reescreve a caixa de texto dos entregadores;
6. agenda uma nova atualização após 500 milissegundos com `root.after(500, self.update)`.

### 4.3.3 Função do arquivo no sistema

Esse arquivo tem papel de monitoramento. Ele não altera a simulação, apenas consome o estado compartilhado e o apresenta de forma legível para o usuário.

## 4.4 `graph.py`

O arquivo `graph.py` foi criado para a etapa experimental da atividade, permitindo medir tempo de execução e calcular speedup.

### 4.4.1 Configuração atual

Na versão atual do script:

- a carga total do experimento foi fixada em 120 pizzas;
- as configurações testadas são 1 chef + 1 entregador, 2 chefs + 2 entregadores, 4 chefs + 4 entregadores e 8 chefs + 8 entregadores;
- a quantidade de pizzas por chef é calculada a partir dessa carga total, resultando em 120, 60, 30 e 15 pizzas por chef, respectivamente;
- o tempo de execução é medido com `timeit.default_timer()`;
- o gráfico final é salvo automaticamente no arquivo `grafico_pizzaria.png`;
- o script trata interrupção por teclado com `KeyboardInterrupt`;
- ao final, o terminal exibe uma tabela textual com configuração, total de threads, pizzas por chef, pizzas totais, tempo e speedup.

### 4.4.2 Fluxo do script

O script realiza os seguintes passos:

1. define uma lista fixa de configurações de teste no formato $(chefs, entregadores)$;
2. calcula automaticamente quantas pizzas cada chef deverá produzir, mantendo o total de 120 pizzas em todas as execuções;
3. reinicializa o estado compartilhado antes de cada teste;
4. imprime no terminal uma mensagem de início para cada experimento;
5. mede o tempo total da simulação;
6. armazena o tempo obtido;
7. calcula o speedup em relação ao primeiro teste, que é a configuração base 1 chef + 1 entregador;
8. imprime uma tabela final com os valores exatos medidos;
9. gera dois subgráficos com o `matplotlib`;
10. salva a figura em disco e tenta exibi-la na tela.

### 4.4.3 Gráficos gerados

Os gráficos produzidos são:

- tempo total versus número total de threads;
- speedup versus número total de threads.

#### 4.4.4 Observação sobre aderência ao enunciado

O arquivo está aderente ao enunciado da Etapa 3, pois executa diretamente os pares $(1, 1)$, $(2, 2)$, $(4, 4)$ e $(8, 8)$, permitindo preencher a tabela do relatório e gerar o gráfico solicitado pelo professor.

# 5 Evidências da Implementação

As figuras abaixo foram reservadas para as capturas de tela solicitadas no enunciado.

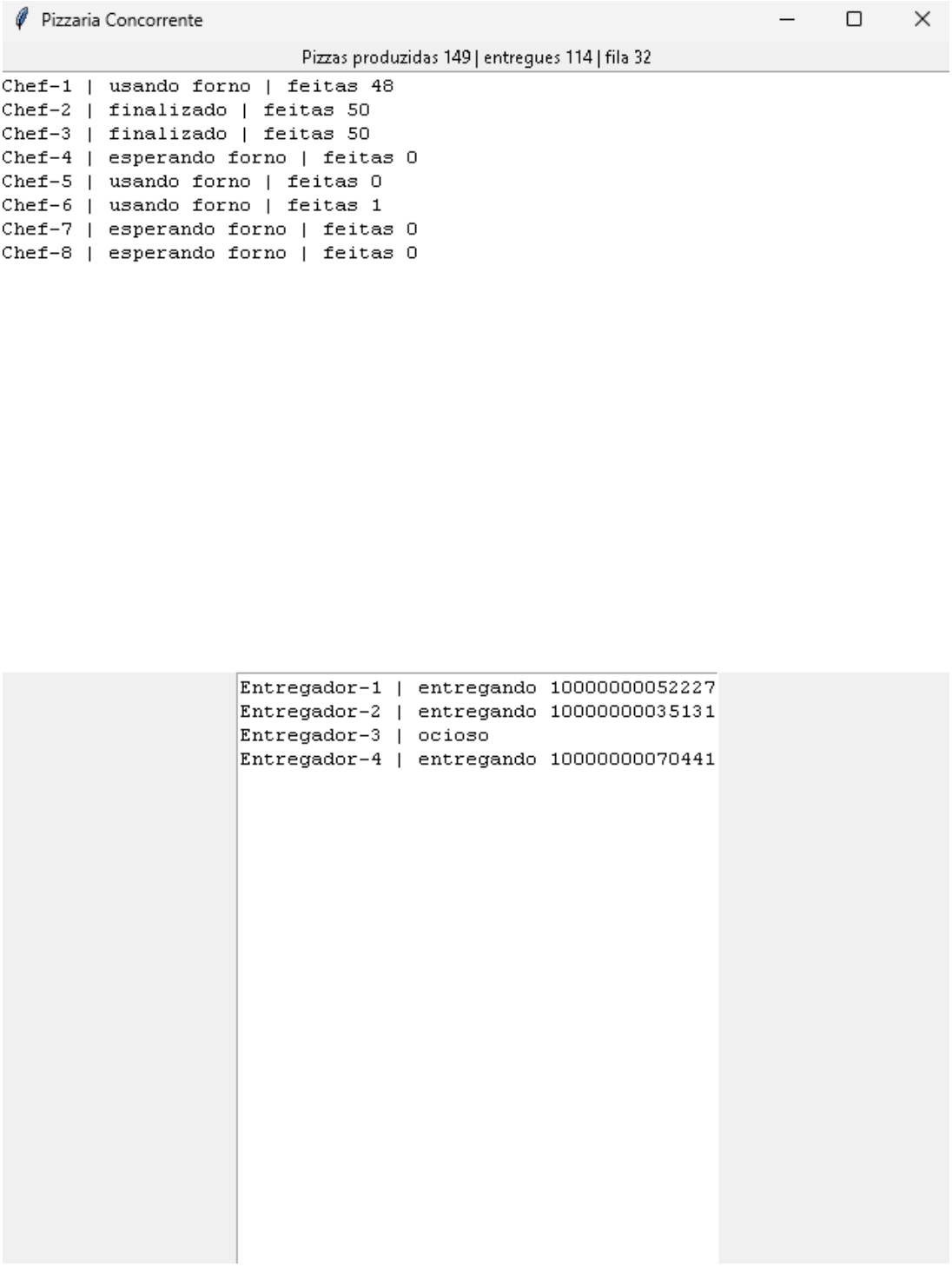


Figura 1: Execução geral do sistema mostrando chefs, entregadores e a interface gráfica em funcionamento.

```
Chef-1 | finalizado | feitas 45
Chef-2 | finalizado | feitas 45
Chef-3 | finalizado | feitas 45
Chef-4 | esperando forno | feitas 0
Chef-5 | usando forno | feitas 5
Chef-6 | esperando forno | feitas 0
Chef-7 | usando forno | feitas 2
Chef-8 | usando forno | feitas 4
```

Figura 2: Situação em que chefs estão aguardando o forno, evidenciando o bloqueio causado pelo semáforo.

```
Chef Chef-6 faz uma pizza sendo o numero 10000000082239
Chef Chef-4 faz uma pizza sendo o numero 10000000003999
Entregador Entregador-1 entrega pizza premium: 10000000082239
```

Figura 3: Situação em que uma pizza fica pronta e ocorre a notificação do consumidor via `Condition`.

# 6 Etapa 3 – Medição e Gráfico

O arquivo `graph.py` foi desenvolvido para automatizar a etapa de medição. A ideia central é executar o programa com diferentes quantidades de threads, registrar o tempo total e comparar o ganho de desempenho.

Na versão atual, as medições são realizadas nas quatro configurações pedidas pelo PDF: 1 chef + 1 entregador, 2 chefs + 2 entregadores, 4 chefs + 4 entregadores e 8 chefs + 8 entregadores.

Para que o gráfico de speedup represente de fato o efeito do paralelismo, todas as configurações processam a mesma carga total: 120 pizzas. Assim, a diferença entre os testes passa a ser a quantidade de threads utilizadas, e não o volume total de trabalho realizado.

Além da exibição em janela quando o backend do `matplotlib` permite, o script salva automaticamente a imagem do resultado em `grafico_pizzaria.png`. Isso facilita a inclusão do gráfico no relatório mesmo em ambientes onde a interface gráfica do `matplotlib` não abre corretamente. O programa também imprime no terminal uma tabela final com os valores exatos, o que ajuda no preenchimento da tabela do relatório.

## 6.1 Métrica utilizada

O tempo total de execução é medido em segundos, e o speedup é calculado pela expressão:

$$S(N) = \frac{T(1)}{T(N)}$$

em que $T(1)$ representa o tempo da configuração de referência e $T(N)$ representa o tempo medido para uma configuração com mais threads.

Nesse experimento, $T(1)$ corresponde ao caso base com 1 chef e 1 entregador processando as mesmas 120 pizzas totais que serão distribuídas nas demais configurações.

## 6.2 Tabela de resultados

| Configuração | Total de threads | Tempo total (s) | Speedup |
|---|---|---|---|
| 1 chef + 1 entregador | 2 | 25.4753 | 1.0000 |
| 2 chefs + 2 entregadores | 4 | 14.5719 | 1.7482 |
| 4 chefs + 4 entregadores | 8 | 10.2249 | 2.4915 |
| 8 chefs + 8 entregadores | 16 | 9.6462 | 2.6410 |

Tabela 2: Tabela para registro dos tempos e speedups.

Os resultados mostram que houve ganho de desempenho com o aumento do número de threads, mas sem crescimento linear perfeito. O melhor resultado observado foi na configuração com 8 chefs e 8 entregadores, que alcançou speedup de 2.6410 com 16 threads no total.

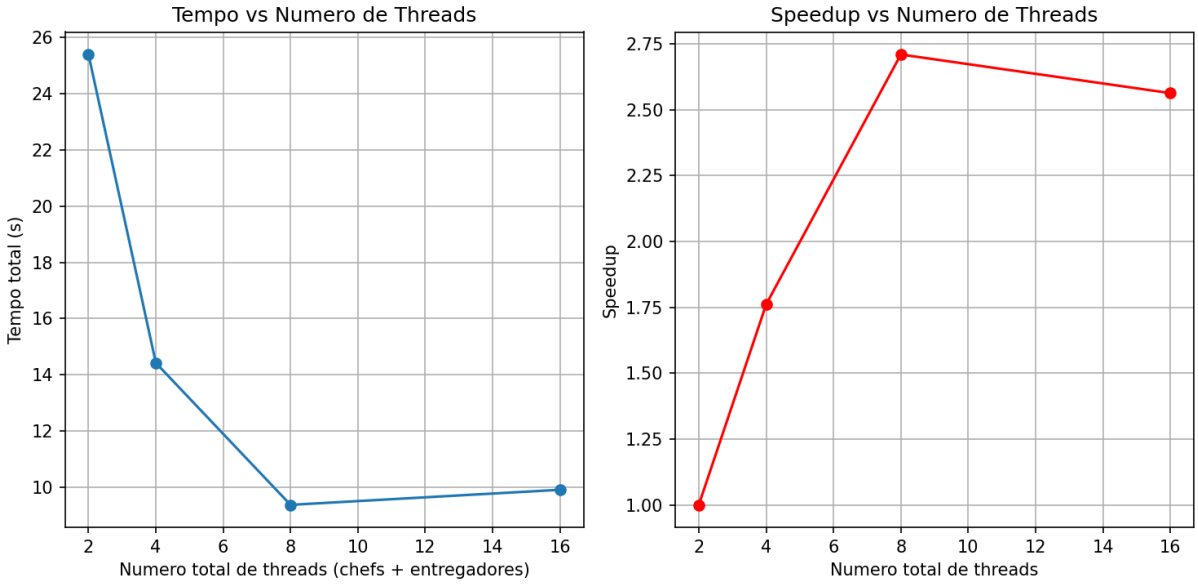


Figura 4: Gráfico de tempo total e speedup gerado a partir do arquivo `graph.py`.

# 7 Etapa 4 – Análise pela Lei de Amdahl

A Lei de Amdahl permite interpretar por que o speedup não cresce indefinidamente com o aumento do número de threads. Mesmo quando há paralelismo, ainda existe uma fração sequencial do programa, além de custos extras de sincronização, contenção e troca de contexto.

A fórmula usada para estimar o limite teórico é:

$$S(N) \approx \frac{1}{B + \frac{1-B}{N}}$$

em que $B$ representa a fração sequencial do programa.

## 7.1 Respostas da Etapa 4

1. **Observando seus resultados, o speedup continua crescendo linearmente até 8 threads? Por quê?**

   Não. O speedup cresce com o aumento do número de threads, mas de forma sublinear. Pelos resultados obtidos, o speedup foi de 1.7482 com 4 threads totais, 2.4915 com 8 threads totais e 2.6410 com 16 threads totais. Isso mostra que houve ganho com paralelismo, porém com retorno decrescente. O crescimento deixa de ser linear porque parte do programa continua sequencial e porque surgem custos extras de sincronização, contenção no semáforo do forno, acesso à fila compartilhada e troca de contexto entre threads.

2. **Estime aproximadamente a fração sequencial $B$ usando dois pontos da tabela.**

   Aplicando a Lei de Amdahl, $S(N) \approx \frac{1}{B+\frac{1-B}{N}}$. Para $N = 8$ threads totais e $S(8) = 2.4915$, obteve-se aproximadamente $B \approx 0.316$. Para $N = 16$ threads totais e $S(16) = 2.6410$, obteve-se aproximadamente $B \approx 0.337$. Fazendo uma estimativa média, temos $B \approx 0.33$. Isso indica que cerca de um terço do comportamento total da aplicação permanece sequencial ou não se beneficia plenamente do paralelismo.

3. **Se você tivesse um computador com 32 núcleos, qual seria o speedup máximo teórico esperado?**

   Considerando $B \approx 0.33$, o speedup teórico para 32 núcleos seria:

   $$S(32) \approx \frac{1}{0.33 + \frac{1-0.33}{32}} \approx 2.85$$

   Portanto, mesmo com muito mais núcleos disponíveis, o ganho não cresceria indefinidamente. O limite teórico absoluto, quando $N \to \infty$, seria aproximadamente $\frac{1}{0.33} \approx 3.03$, o que reforça a ideia de saturação do speedup.

4. **Cite pelo menos duas causas reais de overhead que apareceram na sua implementação.**

   Entre os principais overheads observados estão o GIL do Python, que limita o paralelismo efetivo em trechos CPU-bound como o teste de primalidade, e a contenção no semáforo do forno, que restringe quantos chefs podem trabalhar simultaneamente. Também houve custo de sincronização com `Queue` e `Condition`, além de troca de contexto entre muitas threads concorrentes. Esses fatores ajudam a explicar por que o speedup não cresce de forma linear.

# 8 Etapa 5 – Reflexão Final

O projeto mostra, de maneira prática, que concorrência e paralelismo não significam ganho ilimitado de desempenho. Aumentar o número de threads pode melhorar a vazão até certo ponto, mas também introduz custos de coordenação e disputa por recursos limitados.

## 8.1 Resposta final

A principal lição desta atividade foi perceber que aumentar o número de threads não significa, automaticamente, obter ganho proporcional de desempenho. Na prática, o paralelismo sempre encontra limites impostos por trechos sequenciais, sincronização, contenção por recursos compartilhados e custos de gerenciamento das threads. O experimento da pizzaria mostrou isso claramente: o speedup aumentou até certo ponto, mas depois os ganhos passaram a diminuir.

Outro aprendizado importante foi entender que medir desempenho exige comparar as mesmas condições de trabalho. Quando a carga total foi mantida fixa, o gráfico passou a representar corretamente o efeito do paralelismo. Isso mostrou que a análise de performance não depende apenas do código, mas também da forma como o experimento é planejado.

Esse tipo de saturação do speedup é especialmente crítico em sistemas reais como servidores web de alta concorrência, aplicações de processamento de dados, jogos e sistemas de machine learning. Nesses cenários, muitas requisições ou tarefas competem por CPU, memória, filas e regiões críticas. Quando o projeto não considera esses gargalos, adicionar mais threads ou mais núcleos pode trazer ganho pequeno, ou até piorar o desempenho. Por isso, a principal conclusão é que paralelizar bem significa equilibrar trabalho, reduzir overhead e conhecer os limites reais da arquitetura usada.

# 9 Conclusão

O projeto está bem organizado em módulos e separa claramente as responsabilidades de inicialização, concorrência, interface e medição experimental. O arquivo `main.py` coordena a execução, `pizzaria.py` implementa o comportamento concorrente central, `interface.py` fornece visibilidade sobre o estado interno e `graph.py` viabiliza a análise de desempenho.

Com isso, o código atende à proposta didática da atividade e fornece uma base sólida tanto para a demonstração da concorrência quanto para a análise dos limites práticos do paralelismo.